

1. WATER JUG PROBLEM

Problem

There are two unmarked water jugs:

- 4-gallon jug
- 3-gallon jug

There is an unlimited water supply. We need to obtain exactly 2 gallons in the 4-gallon jug.

Initial State

Both jugs are empty.

Initial state = (0, 0)

Here, (x, y) means:

- x = water in 4-gallon jug
- y = water in 3-gallon jug

Goal State

We need exactly 2 gallons in the 4-gallon jug.

Goal = (2, y)

Operators

The following operations are allowed:

- Fill the 4-gallon jug.
- Fill the 3-gallon jug.
- Empty the 4-gallon jug.
- Empty the 3-gallon jug.
- Pour water from 4-gallon jug to 3-gallon jug.
- Pour water from 3-gallon jug to 4-gallon jug.

STEP	OPERATION	STATE
Initial	Both empty	(0,0)
1	Fill 4-gallon jug	(4,0)
2	Pour 4-gallon into 3-gallon	(1,3)
3	Empty 3-gallon jug	(1,0)
4	Pour 1 gallon into 3-gallon jug	(0,1)
5	Fill 4-gallon jug	(4,1)

6	Pour from 4-gallon into 3-gallon until 3-gallon is full	(2,3)
---	---	-------

Final State

(2,3)

Therefore, the 4-gallon jug contains **exactly 2 gallons**.

Conclusion

The water-jug problem can be solved using **state-space search**, where each possible quantity of water represents a state and each filling, emptying, or pouring operation represents an operator.

2. MARS ROVER PROBLEM

Problem

A Mars Rover must explore the surface of Mars, collect samples, analyze them, avoid obstacles and send useful information back to Earth.

Problem Formulation

Initial State:

Rover is located at a starting position on Mars.

Goal State:

Explore the assigned region and collect/analyse useful samples while safely operating.

Percepts:

Its percepts include:

- Images of the terrain
- Rocks and soil
- Obstacles
- Temperature
- Atmospheric conditions
- Rover position
- Battery level
- Sample composition

- Distance from objects

the rover receives information through sensors.

Environment Characteristics

PROPERTY	MARS ENVIRONMENT
Observable	Partially observable
Deterministic	Partially stochastic
Sequential	Yes
Dynamic	Yes
Discrete/Continuous	Mostly continuous
Agents	Mainly single-agent
Known	Partially unknown

Actions

The rover can

- Move forward
- Move backward
- Turn
- Stop
- Take photographs
- Collect samples
- Drill rocks
- Analyse samples
- Avoid obstacles
- Send information to Earth

Performance Measure

The rover's performance can be measured by:

- Amount of useful scientific information obtained
- Number of samples collected
- Accuracy of analysis
- Area explored
- Energy consumed
- Time taken

- Safety
- Successful communication

Suitable Agent

A model-based utility-based agent is suitable.

Reason

The rover cannot observe the complete Martian environment. It must maintain an internal model and make decisions based on:

- Current information
- Previous observations
- Scientific goals
- Energy availability
- Safety
- Distance

Conclusion

The Mars Rover is an intelligent agent because it perceives its environment, processes information, selects actions and evaluates the results.

3. 8-QUEENS PROBLEM

Problem Statement

The objective of the 8-Queens problem is to place 8 queens on an 8×8 chessboard such that no two queens attack each other.

A queen can attack another queen if they are in the same:

- Row
- Column
- Diagonal

Therefore, a valid solution must satisfy all three conditions.

Problem Formulation

1. Initial State

The chessboard is initially empty.

Initial state = no queens placed

2. State Representation

One convenient representation is:

$[r_1, r_2, r_3, \dots, r_8]$

where r_i represents the row occupied by the queen in column i .

For example:

$[1, 5, 8, 6, 3, 7, 2, 4]$

represents:

- Column 1 $\rightarrow$ Row 1
- Column 2 $\rightarrow$ Row 5
- Column 3 $\rightarrow$ Row 8
- Column 8 $\rightarrow$ Row 4

3. Actions

An action consists of placing a queen in an available row of the next column.

For example:

```
Place queen in column 1
Place queen in column 2
Place queen in column 3
...
Place queen in column 8
```

A placement is allowed only when the new queen does not attack any previously placed queen.

4. Transition Model

When a queen is placed in column c and row r , the state changes by adding r to the configuration.

For example:

$[]$

$\downarrow$

[1]

↓

[1,5]

↓

[1,5,8]

The search continues until eight queens are placed.

5. Goal Test

A state is a goal if:

- Exactly 8 queens have been placed.
- No two queens share the same row.
- No two queens share the same column.
- No two queens share a diagonal.

For two queens at (r1, c1) and (r2, c2), they are on the same diagonal if:

$$|r1 - r2| = |c1 - c2|$$

Search Strategy

The 8-Queens problem can be solved using Depth-First Search (DFS) with backtracking.

Backtracking idea

1. Place a queen in the first column.
2. Move to the next column.
3. Try a safe row.
4. Continue placing queens.
5. If no safe position exists, go back to the previous column.
6. Move that queen to another possible row.
7. Continue until all eight queens are placed.

Pseudocode

```
NQueens (board, column):  
    if column > 8:  
        return SUCCESS  
  
    for each row from 1 to 8:
```

```

        if position is safe:
            place queen at (row, column)

            if NQueens (board, column + 1) == SUCCESS:
                return SUCCESS

            remove queen from (row, column)

    return FAILURE

```

Example Valid Solution

One valid configuration is:

[1, 5, 8, 6, 3, 7, 2, 4]

This means one queen is placed in each column, with the corresponding row values.

4. OLA CAB BOOKING AS A PROBLEM-SOLVING AGENT

Problem Statement

A customer wants to travel from a source location to a destination using a cab-booking application such as OLA.

The customer can select different cab categories, such as:

- Mini
- Micro
- Sedan
- Shared
- Prime

The system should determine an appropriate cab and route based on the customer's requirements.

Type of Problem-Solving Agent

A suitable agent is a **goal-based agent with utility-based decision making**.

Why?

The main goal is:

Reach the destination successfully.

However, multiple cab choices can achieve the same goal.

The customer may also have preferences such as:

- Low fare
- Short travel time
- Comfortable vehicle
- High availability
- Direct route
- Fewer stops

Therefore, simply achieving the goal is not enough. The system should evaluate available choices using a **utility function**.

For example:

$$\text{Utility} = \text{Comfort} + \text{Availability} - \text{Fare} - \text{Travel Time}$$

The exact weights can depend on the customer's preferences.`

Problem Formulation

Initial State

Customer is at the pickup location.

Goal State

Customer reaches the specified destination.

Actions

The agent can:

1. Obtain pickup location.
2. Obtain destination.
3. Check available cabs.
4. Estimate fares.
5. Estimate travel times.
6. Compare cab categories.
7. Select a cab.
8. Book the cab.
9. Provide route/navigation.
10. Confirm arrival.

Pseudocode

OLA_CAB_BOOKING () :

START

```

Get current_location
Get destination

if current_location == destination:
    Display "Already at destination"
    STOP

available_cabs = SearchAvailableCabs(current_location)

if available_cabs is empty:
    Display "No cabs available"
    STOP

for each cab in available_cabs:
    fare = EstimateFare(cab, current_location, destination)
    time = EstimateTravelTime(cab, current_location, destination)
    comfort = GetComfortLevel(cab)
    availability = CheckAvailability(cab)

    utility[cab] = CalculateUtility(
        fare,
        time,
        comfort,
        availability
    )

selected_cab = SelectCabWithBestUtility(utility)

Display selected_cab
Ask customer for confirmation

if customer confirms:
    Book(selected_cab)
    TrackCab()
    NavigateToDestination()

    when destination reached:
        EndTrip()
        Display fare and trip details

else:
    Display "Booking cancelled"

STOP

```

Agent Architecture

A utility-based goal-oriented agent is appropriate because it must satisfy the destination goal while choosing among competing alternatives.

5.UNIFORM COST SEARCH FOR LOGISTICS DELIVERY

Problem Statement

A logistics company needs to transport a package from a starting warehouse **S** to a destination warehouse **G**.

Each route has an associated cost representing factors such as:

- Fuel consumption
- Travel time
- Road charges
- Other transportation costs

The objective is to find the **least-cost path from S to G**.

This problem can be solved using **Uniform Cost Search (UCS)**.

Problem Formulation

Initial State

S

The package starts at warehouse S.

Goal State

G

The package must reach warehouse G.

Actions

From the current warehouse, select one of the connected warehouses and travel along the corresponding route.

Path Cost

If the route costs are:

$$S \rightarrow A = 4$$

$$A \rightarrow B = 3$$

$$B \rightarrow G = 5$$

then the total cost is:

$$4 + 3 + 5 = 12$$

The UCS algorithm always expands the currently discovered path having the **lowest cumulative cost**.

UCS Evaluation Function

Uniform Cost Search uses:

$$f(n) = g(n)$$

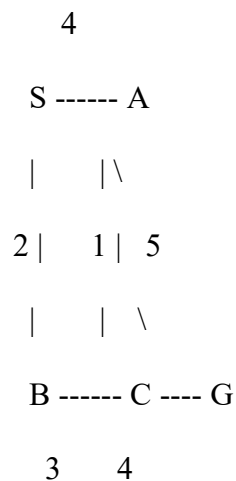
where:

- $g(n)$ = total cost from the starting warehouse S to node n.

Unlike Greedy Best-First Search, UCS does not use only an estimated distance to the destination.

Example Weighted Network

For illustration, consider the following network:



Assume the edges are:

$$S \rightarrow A = 4$$

$$S \rightarrow B = 2$$

$$A \rightarrow C = 1$$

$$A \rightarrow G = 5$$

$$B \rightarrow C = 3$$

$$C \rightarrow G = 4$$

UCS Execution

Step 1

Start at S.

Priority queue:

S: 0

Expand S.

New paths:

B :2

A: 4

Choose the lowest-cost path:

B : 2

Step 2

Expand B.

From B:

$$B \rightarrow C = 3$$

Cumulative cost:

$$2 + 3 = 5$$

Priority queue:

A: 4

C: 5

Choose:

A: 4

Step 3

Expand A.

Possible paths:

$A \rightarrow C = 1$

$A \rightarrow G = 5$

Path to C:

$4 + 1 = 5$

Path to G:

$4 + 5 = 9$

Priority queue:

C: 5

G: 9

Choose C.

Step 4

Expand C.

$C \rightarrow G$ has cost 4.

Therefore:

$5 + 4 = 9$

The destination G can now be reached with cost 9.

Result

Two relevant paths are:

Path 1

$S \rightarrow A \rightarrow G$

Cost:

$$4 + 5 = 9$$

Path 2

$S \rightarrow B \rightarrow C \rightarrow G$

Cost:

$$2 + 3 + 4 = 9$$

Thus, in this example, **both paths have a total cost of 9.**

This also demonstrates an important property of UCS: if multiple paths have the same minimum cost, UCS may return either minimum-cost path depending on the tie-breaking rule.

UCS Pseudocode

UniformCostSearch(start, goal):

 priority_queue $\leftarrow$ empty

 insert (start, 0, [start]) into priority_queue

 best_cost[start] $\leftarrow$ 0

 while priority_queue is not empty:

 current $\leftarrow$ remove node with minimum cost

 if current.node == goal:

 return current.path, current.cost

for each neighbor of current.node:

new_cost =

current.cost + cost(current.node, neighbor)

if neighbor not in best_cost

OR new_cost < best_cost[neighbor]:

best_cost[neighbor] ← new_cost

insert

(neighbor, new_cost, current.path + neighbor)

into priority_queue

return FAILURE

Conclusion

UCS expands the path with the smallest cumulative cost first. If all route costs are non-negative, the first time the goal is selected for expansion, the corresponding path has minimum total cost.